ASSIGNMENT 1

Pointers in C, Matrix Transpose Algorithm, and Recursion in File System Traversal

Name: Mohammad Aleyas

Course Code: CSA0307

Course: Data Structures and Algorithm

Registration Number: 192521220

Q1. What is a Pointer in C? Explain its Relevance

Definition

A pointer in C is a variable that stores the memory address of another variable or of a memory location. Unlike an ordinary variable, which stores a value such as 10 or 25, a pointer stores where that value is located in memory. Pointers are declared by using the asterisk (*) symbol. The address-of operator (&) obtains the address of a variable, while the dereference operator (*) accesses the value stored at the address held by a pointer.

Example:

```
int number = 10;\nint *ptr = &number;\nprintf("%d", *ptr);   // prints 10
```

Here, number contains 10, &number represents its address, and ptr stores that address. The expression *ptr reads the value at that address, so it produces 10.

Relevance and Uses of Pointers

• **Dynamic memory allocation:** Pointers work with functions such as malloc(), calloc(), realloc(), and free() to request and release memory while a program runs. This is useful when the required amount of memory is not known in advance.

• **Pass-by-reference behaviour:** C passes function arguments by value, but a function can receive a pointer and use it to modify the original variable. This is useful for functions that must update values or return multiple results through parameters.

• **Efficient array and string processing:** Array elements are stored in contiguous memory, and pointers can move through these elements efficiently. Character pointers are also commonly used when processing C strings.

• **Building data structures:** Linked lists, trees, graphs, stacks, and queues can use pointers to connect their nodes. This allows structures to grow or change during program execution.

• **Working with functions and system resources:** Function pointers can select functions at runtime, such as callbacks. Pointers are also important in low-level programming, device drivers, and operating-system interfaces.

Precautions when using pointers

Pointers must be used carefully. An uninitialized pointer may contain an invalid address, and dereferencing it can cause undefined behaviour or a crash. A NULL pointer should not be dereferenced. Memory allocated dynamically should be released when it is no longer needed, and a pointer should not be used after its memory has been freed. Therefore, pointers provide power and flexibility, but correct memory management is essential.

Q2. Design an Efficient Algorithm for Matrix Transpose

Meaning of matrix transpose

The transpose of a matrix is formed by converting its rows into columns and its columns into rows. If matrix A has m rows and n columns, its transpose AT has n rows and m columns. The element at row i and column j in A

becomes the element at row j and column i in AT.

For example, if:

```
A =  [1  2  3]\n     [4  5  6]
```

Then its transpose is:

```
AT = [1  4]\n     [2  5]\n     [3  6]
```

Algorithm for an m × n matrix

A straightforward and efficient method for a general rectangular matrix is to create a separate n × m output matrix. Visit each element of the original matrix once and copy it to the reversed row-and-column position in the output.

Pseudocode:

```
TRANSPOSE(A, m, n):\n    create matrix T with n rows and m columns\n    for i = 0 to m - 1:\n
```

Why this algorithm is efficient

The algorithm processes every element exactly once. Since an m × n matrix contains mn elements, the time complexity is O(mn). The output matrix contains mn elements, so the extra space complexity is O(mn). This is an appropriate general solution because a rectangular matrix changes its dimensions after transposition.

In-place transpose for a square matrix

If the matrix is square (n × n), a separate matrix is not necessary. The algorithm can swap A[i][j] with A[j][i] only when j is greater than i. This swaps each pair above and below the main diagonal exactly once.

```
for i = 0 to n - 1:\n    for j = i + 1 to n - 1:\n        swap(A[i][j], A[j][i])
```

This square-matrix method takes $O(n^2)$ time and O(1) auxiliary space because it uses only a temporary variable for swapping. The choice of method depends on the matrix: use the output-matrix method for a general rectangular matrix and the in-place method when a square matrix must be transposed without extra storage.

Q3. Discuss the Use of Recursion in File System Traversal

Introduction

A file system stores information in a hierarchy of directories (folders) and files. A directory can contain files as well as other directories, which may contain further subdirectories. Recursion is a programming technique in which a function calls itself to solve smaller versions of a problem. It is well suited to file system traversal because each directory can be processed using the same steps as the directory above it.

How recursive traversal works

A recursive traversal function receives the path of a directory. It lists the entries in that directory and processes each entry. If an entry is a file, the program can display its name or perform an operation on it. If an entry is a subdirectory, the function calls itself with that subdirectory's path. The calls continue until the program reaches directories with no subdirectories to explore. This stopping condition is the base case.

Pseudocode:

```
TRAVERSE(directory):\n    for each entry in directory:\n        if entry is a regular file:\n
```

Example

Consider this folder structure:

```
College/\n  Notes.txt\n  Programming/\n    C.pdf\n    Java/\n      Basics.docx\n  Photos/\n    Camp
```

Starting at College, the function processes Notes.txt, then enters Programming. Inside Programming it processes C.pdf and enters Java, where it processes Basics.docx. After finishing Java, it returns to Programming, then visits Photos and processes Campus.jpg. The function eventually returns after all reachable entries have been processed.

Advantages and applications

• Simple and readable: the code closely matches the natural tree structure of directories.

• Complete exploration: it can visit files and subdirectories at arbitrary depths without needing to write a separate set of loops for each level.

• Useful for many tasks: recursive traversal supports searching for a file, calculating folder sizes, listing directory contents, finding file types, and applying batch operations.

• Natural backtracking: when a recursive call finishes exploring a subdirectory, execution returns to the parent directory and continues with the next entry.

Limitations and precautions

Very deeply nested directories can create many recursive function calls and may exhaust the call stack. In such cases, an explicit stack or iterative traversal may be safer. The program should handle permission errors, files that disappear during traversal, symbolic links that may create cycles, and inaccessible directories. A visited-path or visited-inode set can help avoid revisiting the same directory through links. The traversal should also avoid following symbolic links unless that behaviour is intended.

Conclusion

Recursion is a natural and effective technique for traversing hierarchical file systems because every directory can be handled using the same function. With a clear base case and appropriate error and link handling, recursive traversal makes directory searching and file management programs easier to design and maintain.